Image compression by discrete cosine transform

Noa Desjouis (49118)

with Joris Collomb-Patton (52020)

2026

Problematic

How to compress images and videos with the JPEG/MPEG algorithms?

I Change of basis

II Huffman's encoding

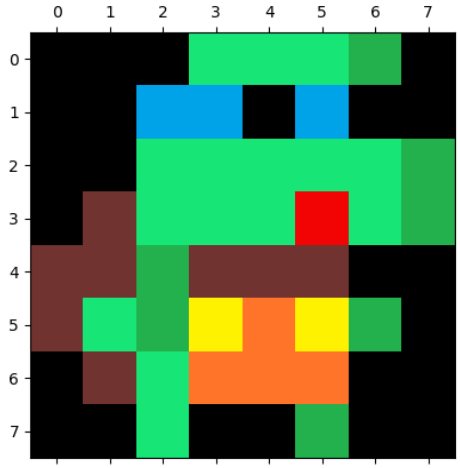
III video Compression





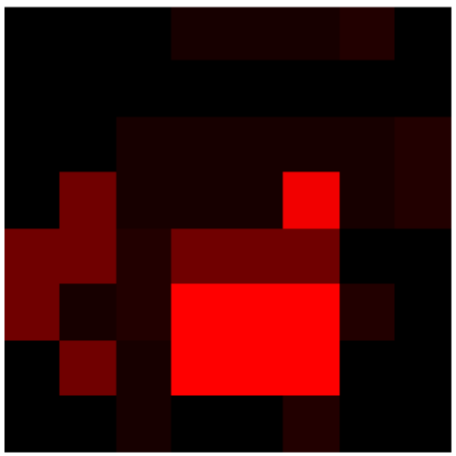
Image compression : Image encoding

Les images sont encodées par trois composantes :



Stockage pour une image 1000×1000 :
$1000 \times 1000 \times 3 = 3\text{Mo}$

Composante rouge



Composante verte



Composante bleue

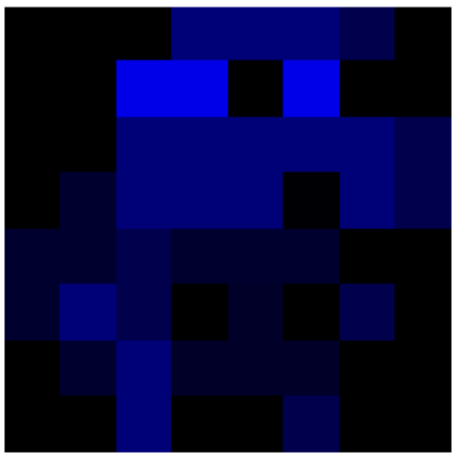
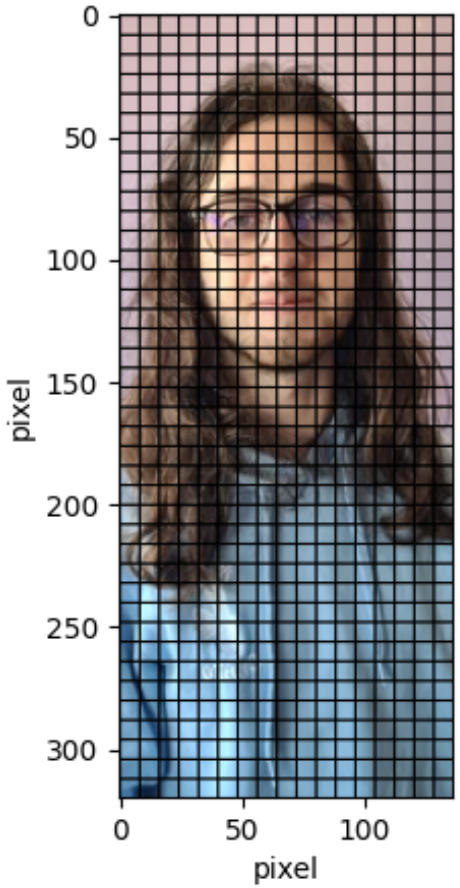
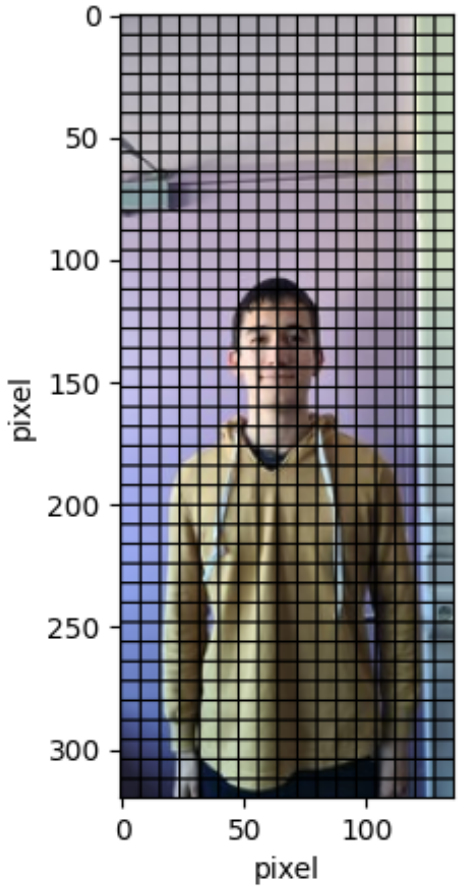


Image compression : Sampling





The matrix is sampled into 8×8 blocs.

Image compression : Change of basis

Coordinates in the usual base

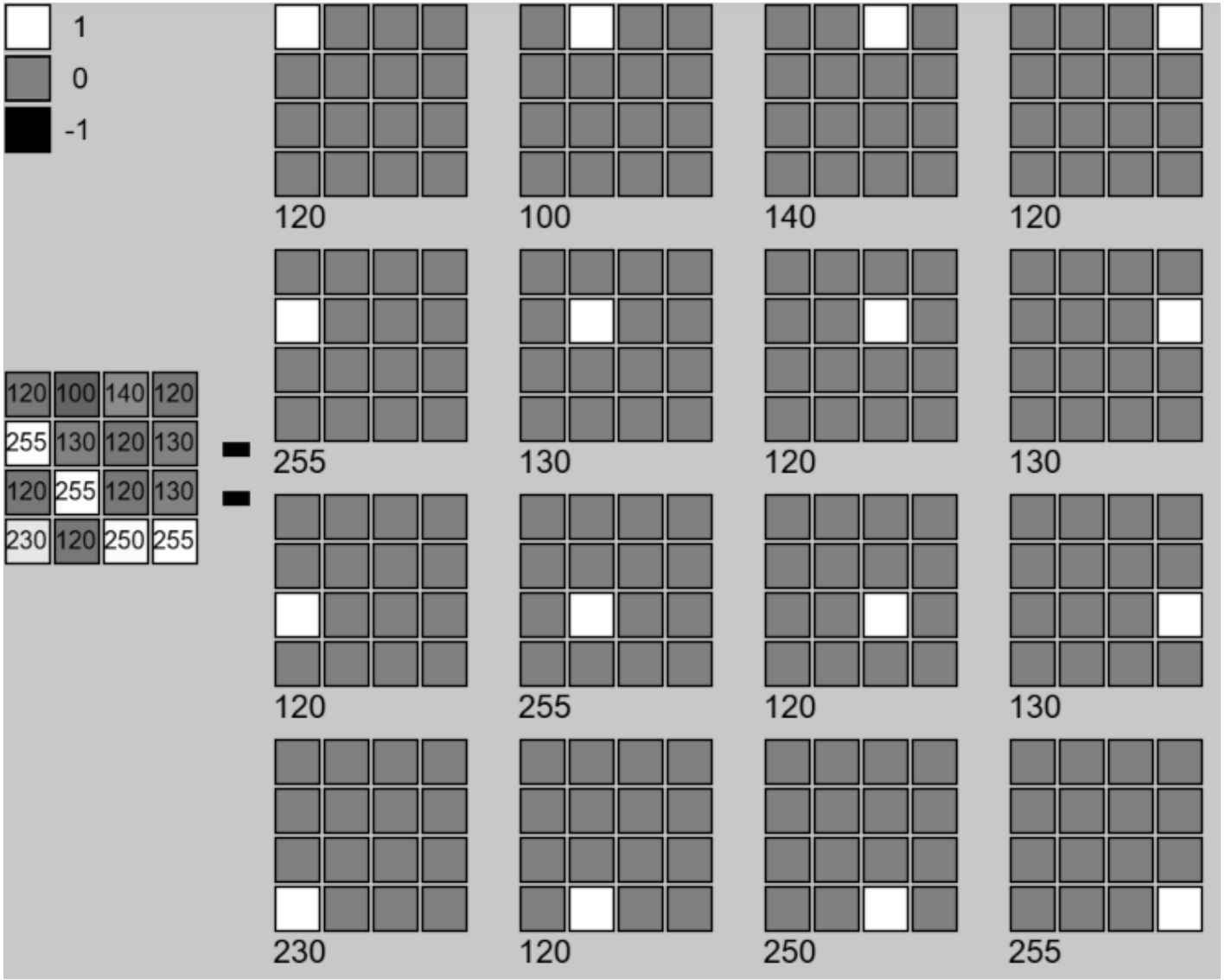


Image compression : Change of basis

Coordinates in the cosine base

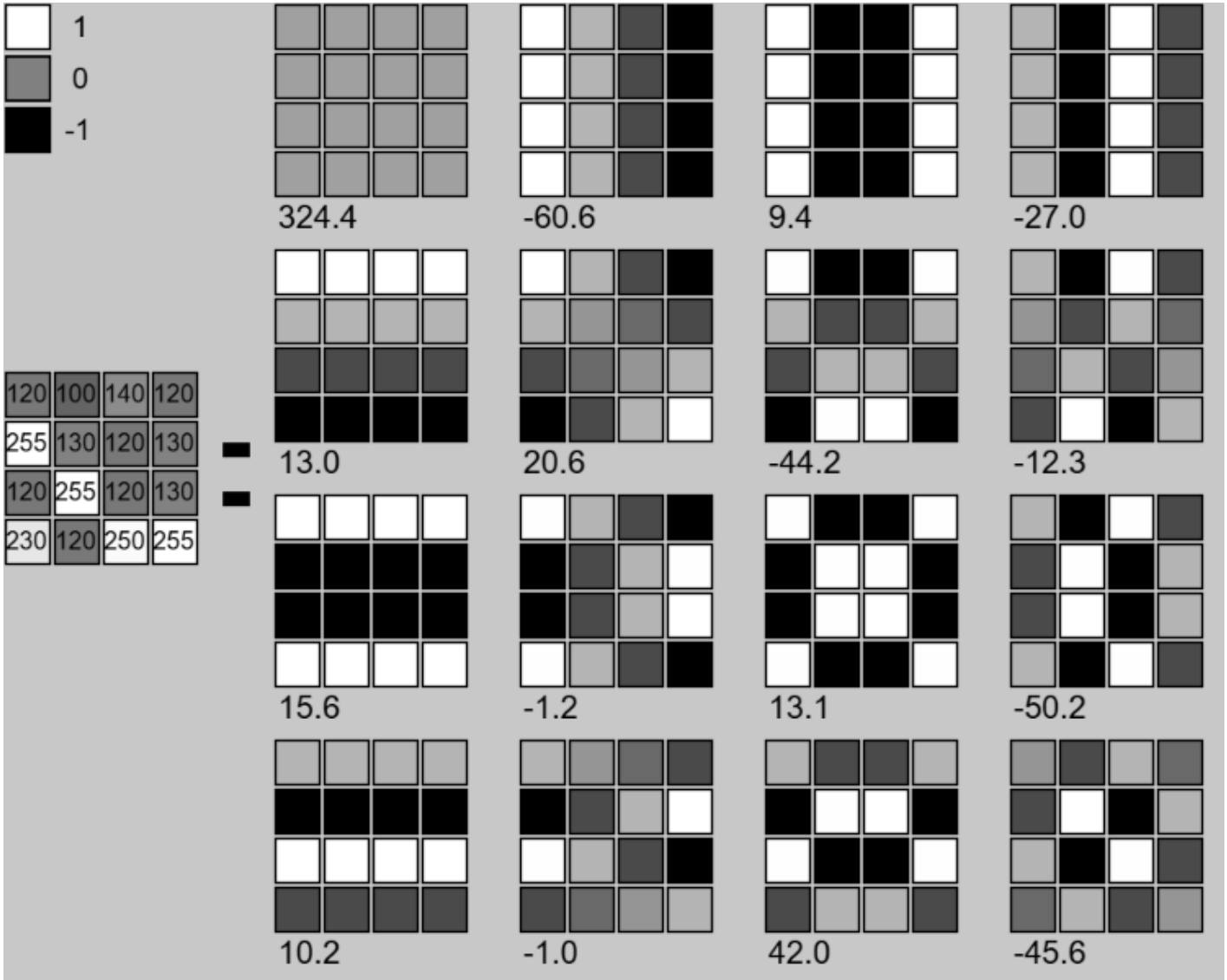
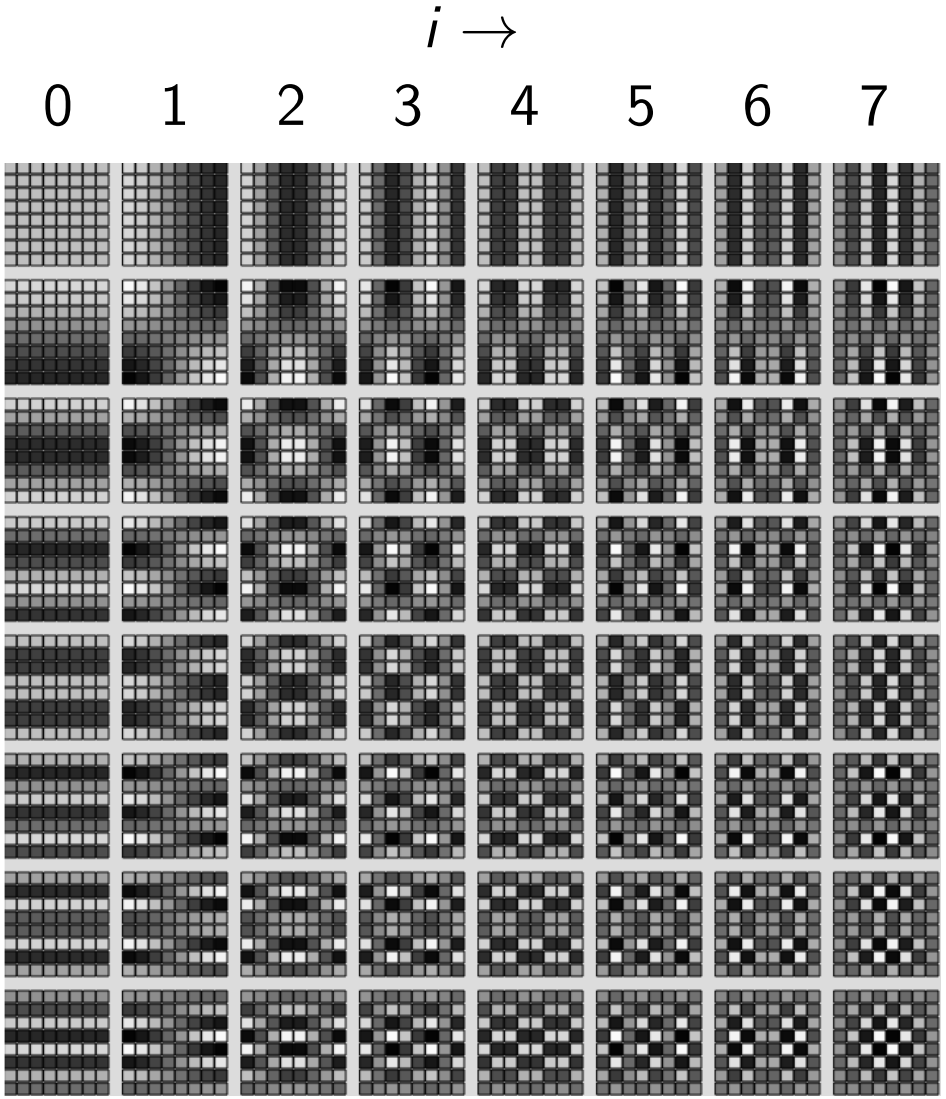


Image compression : Discrete cosine transform



$$C(x) = \begin{cases} \frac{1}{\sqrt{2}} & \text{if } x = 0 \\ \text{else } 1 \end{cases}$$

$$Bloc_{dct}[x][y] = \frac{1}{4} \cdot C(x) \cdot C(y) \cdot \sum_{i=0}^{7} \sum_{j=0}^{7} Bloc_{RGB}[i][j] \cdot \cos\left(\frac{i(2x+1)}{16}\right) \cdot \cos\left(\frac{j(2y+1)}{16}\right)$$

Image compression : Quantization

$$M = \begin{bmatrix} m_{0,0} & m_{0,1} & \cdots & m_{0,7} \\ m_{1,0} & m_{1,1} & \cdots & m_{1,7} \\ \vdots & \vdots & \ddots & \vdots \\ m_{7,0} & m_{7,1} & \cdots & m_{7,7} \end{bmatrix}$$

JPEG's luminance quantization matrix

$$Q = \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}$$

$$S = \begin{bmatrix} m_{0,0} \div 16 & m_{0,1} \div 11 & m_{0,2} \div 10 & m_{0,3} \div 16 & m_{0,4} \div 24 & m_{0,5} \div 40 & m_{0,6} \div 51 & m_{0,7} \div 61 \\ m_{1,0} \div 12 & m_{1,1} \div 12 & m_{1,2} \div 14 & m_{1,3} \div 19 & m_{1,4} \div 26 & m_{1,5} \div 58 & m_{1,6} \div 60 & m_{1,7} \div 55 \\ m_{2,0} \div 14 & m_{2,1} \div 13 & m_{2,2} \div 16 & m_{2,3} \div 24 & m_{2,4} \div 40 & m_{2,5} \div 57 & m_{2,6} \div 69 & m_{2,7} \div 56 \\ m_{3,0} \div 14 & m_{3,1} \div 17 & m_{3,2} \div 22 & m_{3,3} \div 29 & m_{3,4} \div 51 & m_{3,5} \div 87 & m_{3,6} \div 80 & m_{3,7} \div 62 \\ m_{4,0} \div 18 & m_{4,1} \div 22 & m_{4,2} \div 37 & m_{4,3} \div 56 & m_{4,4} \div 68 & m_{4,5} \div 109 & m_{4,6} \div 103 & m_{4,7} \div 77 \\ m_{5,0} \div 24 & m_{5,1} \div 35 & m_{5,2} \div 55 & m_{5,3} \div 64 & m_{5,4} \div 81 & m_{5,5} \div 104 & m_{5,6} \div 113 & m_{5,7} \div 92 \\ m_{6,0} \div 49 & m_{6,1} \div 64 & m_{6,2} \div 78 & m_{6,3} \div 87 & m_{6,4} \div 103 & m_{6,5} \div 121 & m_{6,6} \div 120 & m_{6,7} \div 101 \\ m_{7,0} \div 72 & m_{7,1} \div 92 & m_{7,2} \div 95 & m_{7,3} \div 98 & m_{7,4} \div 112 & m_{7,5} \div 100 & m_{7,6} \div 103 & m_{7,7} \div 99 \end{bmatrix}$$

Image compression : Results

Image originale



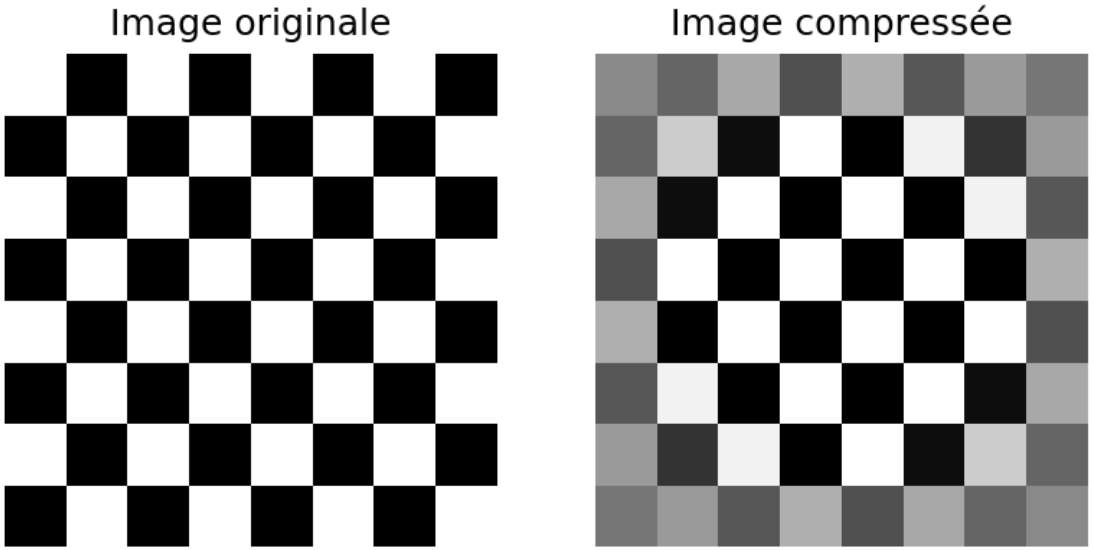
Image compressée



Image trop compressée



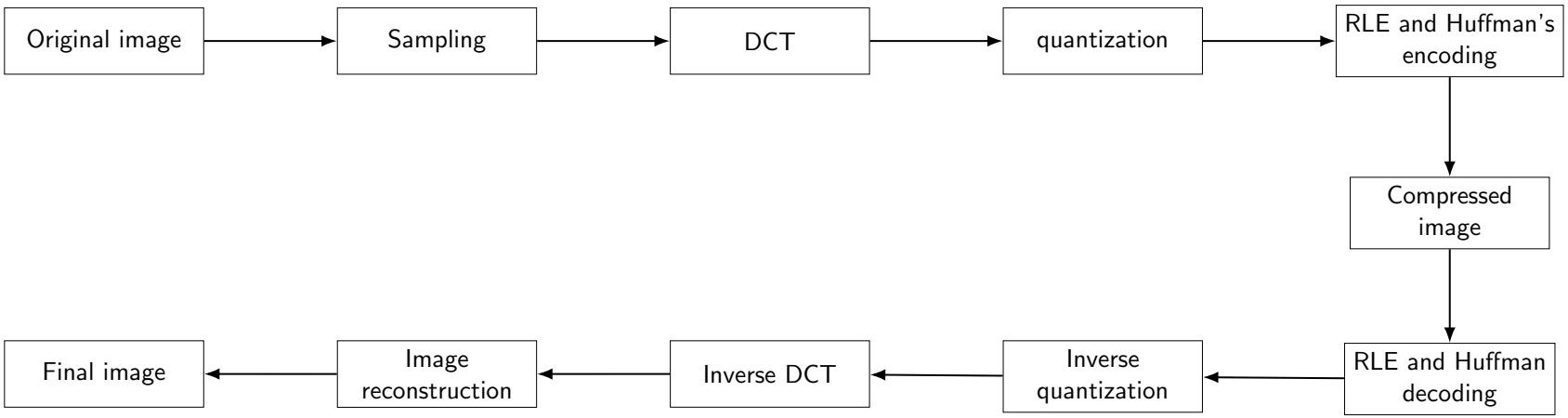
Image compression : Limits



Excessive compression of a black and white matrix

When high frequency cosines are the most important, the loss is very noticeable.

Huffman's encoding



Huffman's encoding : Principle

original matrix

$$\begin{bmatrix} 206 & 215 & 224 & 230 & 235 & 237 & 238 & 238 \\ 194 & 202 & 211 & 223 & 233 & 238 & 238 & 238 \\ 199 & 210 & 222 & 233 & 238 & 239 & 239 & 239 \\ 217 & 228 & 236 & 237 & 237 & 238 & 239 & 239 \\ 221 & 223 & 222 & 221 & 224 & 235 & 240 & 240 \\ 202 & 201 & 201 & 208 & 224 & 238 & 241 & 241 \\ 186 & 191 & 199 & 216 & 237 & 241 & 239 & 241 \\ 185 & 194 & 214 & 235 & 241 & 239 & 239 & 242 \end{bmatrix}$$

final matrix

$$\begin{bmatrix} 49 & -10 & -3 & 0 & 0 & 0 & 0 & 0 \\ 1 & 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & -2 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

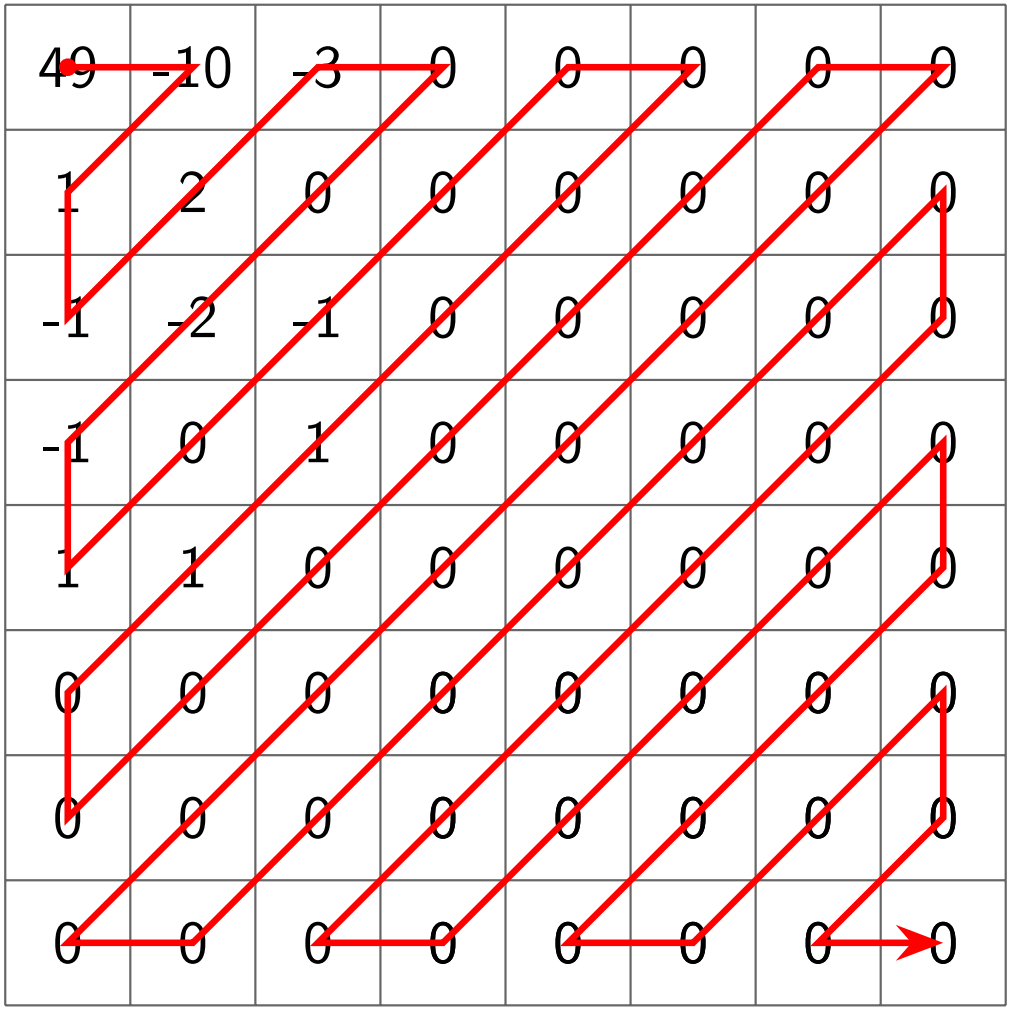
We can notice the 0-redudancy.

Huffman's encoding : Data type

List of type (n,b),v List

- n = number of 0 encoutered before v
- b = number of bits used to encode v
- v = Non 0 value encoutered

Huffman's encoding : Zig-zag path



$[(0,6), 49, (0,1), 1, (0,4), -10, (0,2), -3, (0,2), 2, (0,1), -1,$
(0, 1), -1, (0, 2), -2, (4, 1), -1, (1, 1), 1, (1, 1), 1, (0, 1), 1,
(46, 0)]

Huffman's encoding : v encoding

- v is encoded on b bits
- $b = \lfloor log_2(|b|) \rfloor + 1$

Huffman's encoding : v encoding

- v is encoded on b bits
- $b = \lfloor log_2(|b|) \rfloor + 1$

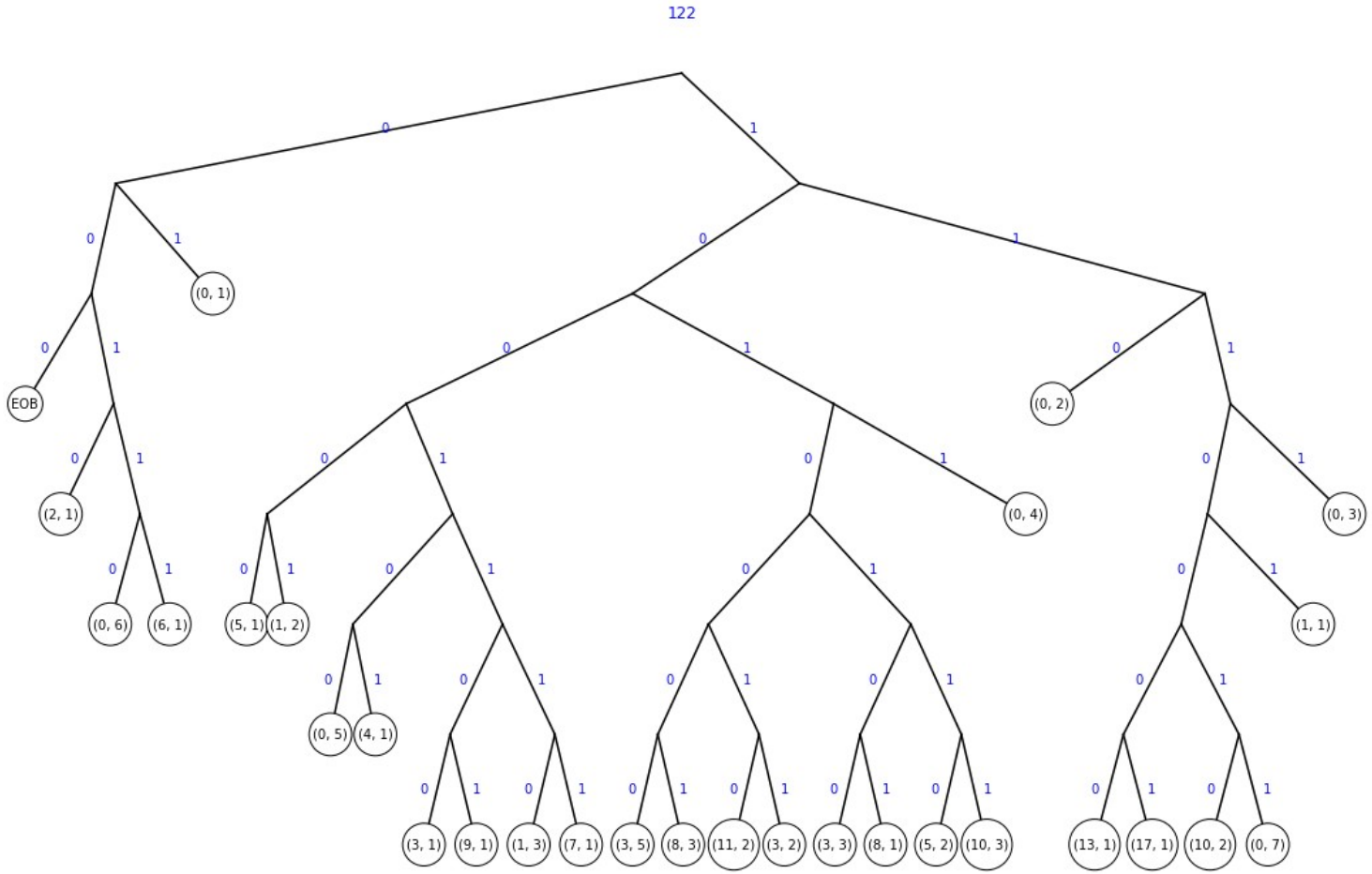
-1	1
0	1

-3	-2	2	3
00	01	10	11

-7	-6	-5	-4	4	5	6	7
000	001	010	011	100	101	110	111

Huffman's encoding : (n,b) encoding

- (n,b) encoded by a Huffman's tree



Huffman's encoding : Application

Image originale



Compression : 1:1

Image compressée



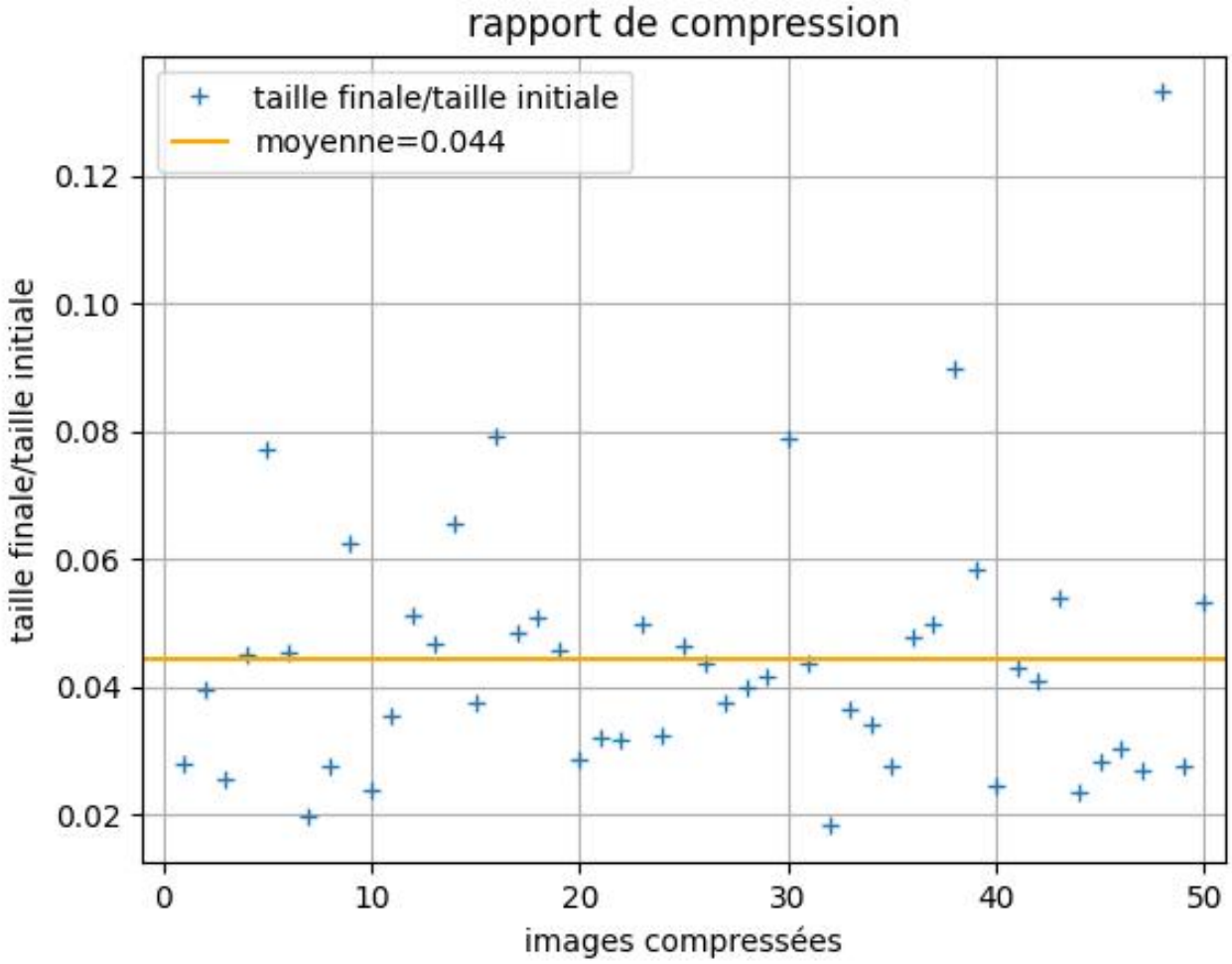
Compression : 20:1

Image trop compressée



Compression : 100:1

Huffman's encoding : Performance



Storage for a 1000×1000 image : $132ko$

Video compression : Naïve compression

for a 2 hour film at 24 frames per second in 1080p resolution

- 178 000 images
- 2 073 600 pixels/image (1920×1080)
- Size after JPEG compression : 100 Go

Video compression : Encoding the difference

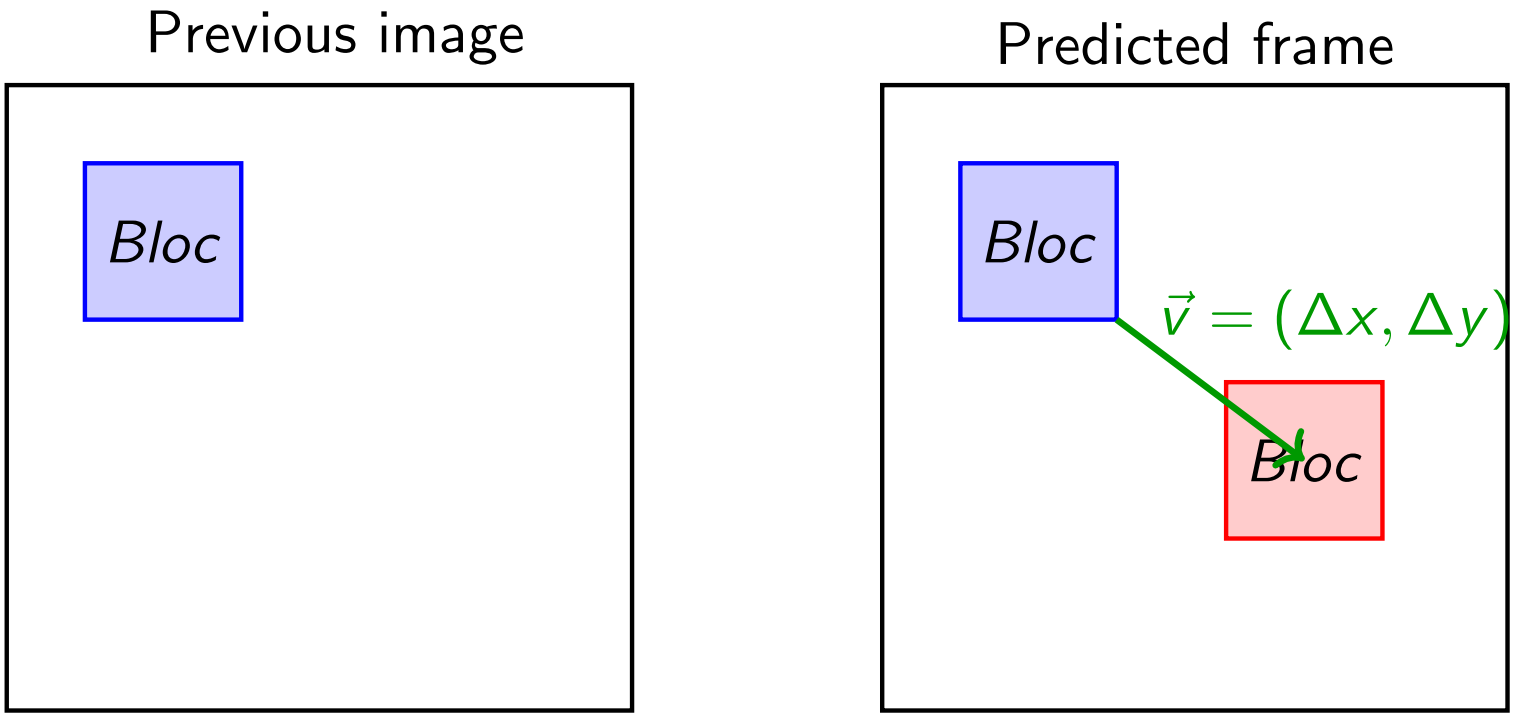
To remedy this issue, we used the temporal continuity of videos.



Predicted frames encoding :

- Movement vector
- Error Matrix

Video compression : Movement vector



Storage for each 8×8 bloc of a movement vector.

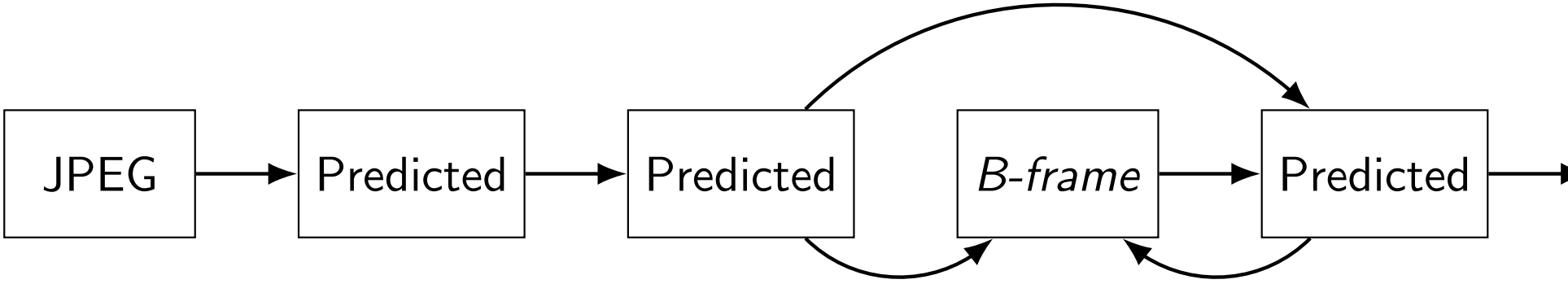
Conclusion

We have :

- Implemented the JPEG compression algorithm with an average compression ratio of 25:1.
- Studied its use in MPEG compression.

Annexe : *B-frames*

To further reduce the memory used :



B-frames encoding :

- Previous frame's movement vector
- Next frame's movement vector
- Error matrix